Search...

Courses Tutorials Practice Jobs

Problem Editorial Submissions Comments

Output Window

Compilation Results Custom Input

Input:

arr[] =

7 10 4 3 20 15

k =

3

Your Output:

7

Expected Output:

7

Java (21) Start Timer

```java
import java.util.*;

class Solution {
    public static int kthSmallest(int[] arr, int k) {
        Arrays.sort(arr);
        return arr[k-1];
    }
}
```

Custom Input Compile & Run Submit

Search... Courses Tutorials Practice Jobs M

Problem Editorial Submissions Comments

Output Window

Compilation Results Custom Input

Input:

k =

2

arr[] =

1 5 8 10

Your Output:

5

Expected Output:

5

Java (21) Start Timer

```java
import java.util.*;

class Solution {
    int getMinDiff(int[] a, int k) {
        int n = a.length;
        Arrays.sort(a);

        int res = a[n-1] - a[0];

        for (int i = 1; i < n; i++)
            if (a[i] >= k)
                res = Math.min(res,
                    Math.max(a[i-1] + k, a[n-1] - k)
                  - Math.min(a[0] + k, a[i] - k));

        return res;
    }
}
```

Custom Input Compile & Run Submit

Search... Courses Tutorials Practice Jobs M

Problem Editorial Submissions Comments

Output Window

Compilation Results Custom Input

Compilation Completed

Case 1

Input:

arr[] =

1 3 5 8 9 2 6 7 6 8 9

Your Output:

3

Expected Output:

3

Java (21) Start Timer

```java
class Solution {
    static int minJumps(int[] arr) {
        if (arr[0] == 0) return -1;

        int jumps = 1, maxReach = arr[0], steps = arr[0];

        for (int i = 1; i < arr.length; i++) {
            if (i == arr.length - 1) return jumps;

            maxReach = Math.max(maxReach, i + arr[i]);
            steps--;

            if (steps == 0) {
                jumps++;
                if (i >= maxReach) return -1;
                steps = maxReach - i;
            }
        }
        return -1;
    }
}
```

Custom Input Compile & Run Submit

Problem List | Submit | 0 | 00:24:14 | Premium

Description | Editorial | Solutions | Submissions

287. Find the Duplicate Number

Solved

Medium | Topics | Companies

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.

There is only **one repeated number** in `nums`, return *this repeated number*.

You must solve the problem **without** modifying the array `nums` and using only constant extra space.

Example 1:

```
Input: nums = [1,3,4,2,2]
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
```

25.4K | 506 | 194 Online

Code

Java | Auto

```java
class Solution {
    public int findDuplicate(int[] nums) {
        int s = nums[0], f = nums[0];
        do {
            s = nums[s];
            f = nums[nums[f]];
        } while (s != f);
```

Saved | Ln 1, Col 1

Testcase | Test Result

[1,3,4,2,2]

Output

2

Expected

2

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Input:

a[] =

1 3 5 7

b[] =

0 2 6 8 9

Your Output:

[0, 1, 2, 3]
[5, 6, 7, 8, 9]

Expected Output:

[0, 1, 2, 3]
[5, 6, 7, 8, 9]

Java (21) | Start Timer

```java
class Solution {
    public void mergeArrays(int[] a, int[] b) {
        int n = a.length, m = b.length;

        for (int g = (n + m + 1) / 2; g > 0; g = (g == 1) ? 0 : (g + 1)
            for (int i = 0, j = g; j < n + m; i++, j++) {
                int x = (i < n) ? a[i] : b[i - n];
                int y = (j < n) ? a[j] : b[j - n];
                if (x > y)
                    if (i < n && j < n) { a[i] = y; a[j] = x; }
                    else if (i < n) { a[i] = y; b[j - n] = x; }
                    else { b[i - n] = y; b[j - n] = x; }
            }
    }
}
```

Custom Input | Compile & Run | Submit

Search... Courses Tutorials Practice Jobs M

Problem Editorial Submissions Comments

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Input:

a[] =

1 3 5 7

b[] =

0 2 6 8 9

Your Output:

[0, 1, 2, 3]
[5, 6, 7, 8, 9]

Expected Output:

[0, 1, 2, 3]
[5, 6, 7, 8, 9]

Java (21) Start Timer

```java
class Solution {
    public void mergeArrays(int[] a, int[] b) {
        int n = a.length, m = b.length;

        for (int g = (n + m + 1) / 2; g > 0; g = (g == 1) ? 0 : (g + 1)
            for (int i = 0, j = g; j < n + m; i++, j++) {
                int x = (i < n) ? a[i] : b[i - n];
                int y = (j < n) ? a[j] : b[j - n];
                if (x > y)
                    if (i < n && j < n) { a[i] = y; a[j] = x; }
                    else if (i < n) { a[i] = y; b[j - n] = x; }
                    else { b[i - n] = y; b[j - n] = x; }
            }
    }
}
```

Custom Input Compile & Run Submit

Description | Editorial | Solutions | Submissions

56. Merge Intervals

Solved

Medium | Topics | Companies

Given an array of `intervals` where `intervals[i] = [start_i, end_i]`, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input.*

Example 1:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].
```

Example 2:

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
Explanation: Intervals [1,4] and [4,5] are considered
```

24.5K | 282 | 497 Online

Code

Java | Auto

```java
import java.util.*;

class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a,b) -> a[0] - b[0]);
        List<int[]> res = new ArrayList<>();

```

Saved | Ln 1, Col 1

Testcase | Test Result

```
[[1,3],[2,6],[8,10],[15,18]]
```

Output

```
[[1,6],[8,10],[15,18]]
```

Expected

```
[[1,6],[8,10],[15,18]]
```

Search... Courses Tutorials Practice Jobs M

Problem Editorial Submissions Comments

Output Window

Compilation Results Custom Input

Compilation Completed

Case 1

Input:

n

5

Your Output:

1 2 0

Expected Output:

1 2 0

Java (21) Start Timer

```java
import java.math.BigInteger;
import java.util.ArrayList;

class Solution {
    static ArrayList<Integer> factorial(int N) {
        BigInteger fact = BigInteger.ONE;

        for (int i = 1; i <= N; i++) {
            fact = fact.multiply(BigInteger.valueOf(i));
        }

        ArrayList<Integer> ans = new ArrayList<>();
        String s = fact.toString();

        for (char c : s.toCharArray()) {
            ans.add(c - '0');
        }

        return ans;
    }
}
```

Custom Input Compile & Run Submit

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Input:

a[] =

11 7 1 13 21 3 7 3

b[] =

11 3 7 1 7

Your Output:

true

Expected Output:

true

Java (21) | Start Timer

```java
import java.util.*;

class Solution {
    public static boolean isSubset(int a[], int b[]) {
        HashMap<Integer, Integer> freq = new HashMap<>();

        // Count elements of a[]
        for (int num : a) {
            freq.put(num, freq.getOrDefault(num, 0) + 1);
        }

        // Check elements of b[]
        for (int num : b) {
            if (!freq.containsKey(num) || freq.get(num) == 0) {
                return false;
            }
            freq.put(num, freq.get(num) - 1);
        }

        return true;
    }
}
```

Custom Input | Compile & Run | Submit

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Input:

arr[] =

1 4 45 6 10 8

target =

13

Your Output:

true

Expected Output:

true

Java (21) | Start Timer

```java
import java.util.Arrays;

class Solution {
    public static boolean hasTripletSum(int[] nums, int target) {
        int n = nums.length;

        Arrays.sort(nums);

        for (int i = 0; i < n - 2; i++) {
            int left = i + 1;
            int right = n - 1;

            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];

                if (sum == target) {
                    return true;
                } else if (sum < target) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return false;
    }
}
```

Custom Input | Compile & Run | Submit

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Compilation Completed

Case 1

Input:

arr[] =

3 0 1 0 4 0 2

Your Output:

10

Expected Output:

10

Java (21) | Start Timer

```java
class Solution {
    public int maxWater(int arr[]) {
        int left = 0, right = arr.length - 1;
        int leftMax = 0, rightMax = 0;
        int water = 0;

        while (left <= right) {
            if (arr[left] <= arr[right]) {
                if (arr[left] >= leftMax) {
                    leftMax = arr[left];
                } else {
                    water += leftMax - arr[left];
                }
                left++;
            } else {
                if (arr[right] >= rightMax) {
                    rightMax = arr[right];
                } else {
                    water += rightMax - arr[right];
                }
                right--;
            }
        }

        return water;
    }
}
```

Custom Input | Compile & Run | Submit